

Banco de Dados

Aula 7 - CREATE TABLE, INSERT, UPDATE, DELETE, DROP e VIEW



MySQL[®]

CREATE TABLE

Na aula anterior vimos como modelar um schema de banco de dados, porém ainda não aprendemos como criar um novo schema e inserir novas tabelas. Além disso, vamos ver outras cláusulas de Data Manipulation Language (DML) e Data Definition Language (DDL) para construir as tabelas, estabelecer relações e alterar dados inseridos no banco a começar pela cláusula CREATE TABLE.

Para criar uma tabela no banco de dados e definir sua estrutura usamos a cláusula CREATE TABLE com a seguinte estrutura:

```
-- Sintaxe básica
CREATE TABLE nome_tabela (
    coluna1 TIPO_DE_DADO CONSTRAINTS,
    coluna2 TIPO_DE_DADO CONSTRAINTS,
    ...
);
```

CREATE TABLE

Observe como as constraints de PRIMARY KEY e FOREIGN KEY são utilizadas para relacionar a tabela professores com a tabela disciplinas.

Além disso temos as constraints `NOT NULL`, `UNIQUE` e `AUTO_INCREMENT`.

```
-- Exemplo: Tabela de Professores
CREATE TABLE professores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    data_contratacao DATE NOT NULL,
    salario DECIMAL(10, 2)
);

-- Com Foreign Key
CREATE TABLE disciplinas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    codigo VARCHAR(20) UNIQUE NOT NULL,
    carga_horaria INT NOT NULL,
    professor_id INT,
    FOREIGN KEY (professor_id) REFERENCES professores(id)
);
```

INSERT

Quando precisamos adicionar novos registros ao banco de dados utilizamos o INSERT. Temos que seguir a estrutura abaixo indicando o nome da tabela, as colunas que receberão novos registros e os valores a serem inseridos

```
-- Sintaxe básica
INSERT INTO nome_tabela (coluna1, coluna2, ... )
VALUES (valor1, valor2, ... );

-- Inserção única
INSERT INTO professores (nome, email, data_contratacao, salario)
VALUES ('Prof. Carlos', 'carlos.p@exemplo.com', '2020-08-15', 7500.00);

-- Inserção múltipla
INSERT INTO estudantes (nome, email, curso, data_matricula, ativo)
VALUES
    ('Ana Silva', 'ana@exemplo.com', 'Informática', '2024-02-01', TRUE),
    ('João Santos', 'joao@exemplo.com', 'Informática', '2024-02-01', TRUE),
    ('Maria Oliveira', 'maria@exemplo.com', 'Administração', '2024-02-15', TRUE);

-- Inserindo com relacionamento (Foreign Key)
INSERT INTO notas (estudante_id, disciplina_id, nota, data_avaliacao)
VALUES (1, 1, 8.5, '2024-03-15');
```

UPDATE

A cláusula UPDATE é utilizada para modificar um registro que já foi inserido anteriormente. Um ponto importante aqui é sempre utilizar o WHERE em conjunto com UPDATE, para evitar alterar todos os registros de uma tabela. Vamos ver os exemplos a lado:

```
-- Sintaxe básica
UPDATE nome_tabela
SET coluna1 = valor1, coluna2 = valor2, ...
WHERE condição;

-- Atualizar salário de um professor específico
UPDATE professores
SET salario = 8000.00
WHERE id = 1;

-- Atualizar múltiplas colunas
UPDATE professores
SET salario = salario * 1.10,
    email = 'carlos.novo@exemplo.com'
WHERE nome = 'Prof. Carlos';

-- Desativar estudante
UPDATE estudantes
SET ativo = FALSE
WHERE id = 4;

-- ⚠ SEM WHERE atualiza TODOS os registros!
UPDATE estudantes SET ativo = TRUE; -- CUIDADO!
```

DELETE

Quando precisamos remover um ou mais registros de uma tabela do banco de dados utilizamos a cláusula DELETE.

```
-- Sintaxe básica
DELETE FROM nome_tabela
WHERE condição;

-- Deletar um estudante específico
DELETE FROM estudantes
WHERE id = 10;

-- Deletar por condição
DELETE FROM notas
WHERE nota < 5.0;
```

DELETE

```
-- Deletar estudantes inativos sem notas
DELETE FROM estudantes
WHERE ativo = FALSE
AND id NOT IN (SELECT DISTINCT estudante_id FROM notas);

-- ⚠ SEM WHERE apaga TODOS os registros!
DELETE FROM estudantes; -- CUIDADO!

-- TRUNCATE: alternativa mais rápida para limpar tabela
TRUNCATE TABLE notas; -- Remove todos os dados e reseta AUTO_INCREMENT
```

DROP

A cláusula DROP remove permanentemente tabelas, schemas ou bancos de dados. ⚠ Esta operação é irreversível!

```
-- Remover tabela
DROP TABLE nome_tabela;

-- Remover tabela se existir (evita erro)
DROP TABLE IF EXISTS notas;

-- Remover schema/banco de dados
DROP SCHEMA IF EXISTS escola_avancada;
DROP DATABASE IF EXISTS escola_avancada;
```


DROP

```
-- ⚠️ ATENÇÃO: Ordem importa com Foreign Keys!
-- Correto:
DROP TABLE notas;           -- Primeiro as tabelas dependentes
DROP TABLE disciplinas;
DROP TABLE estudantes;
DROP TABLE professores;     -- Por último as tabelas referenciadas

-- Alternativa: desabilitar verificação de FK temporariamente
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE professores;
SET FOREIGN_KEY_CHECKS = 1;
```

DROP vs DELETE vs TRUNCATE:

1. DROP remove estrutura
2. DELETE remove dados (mantém estrutura)
3. TRUNCATE limpa dados rapidamente

VIEW

VIEW é uma "tabela virtual" baseada em uma consulta SQL. Facilita consultas complexas e melhora segurança.

```
-- Sintaxe básica
CREATE VIEW nome_view AS
SELECT ... FROM ... WHERE ... ;

-- View: Estudantes com suas médias
CREATE VIEW vw_medias_estudantes AS
SELECT
    e.nome AS estudante,
    e.curso,
    COUNT(n.id) AS total_avaliacoes,
    ROUND(AVG(n.nota), 2) AS media_geral,
    CASE
        WHEN AVG(n.nota) ≥ 7.0 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS status
FROM estudantes e
LEFT JOIN notas n ON e.id = n.estudante_id
GROUP BY e.id;
```

VIEW

```
-- Usando a view
SELECT * FROM vw_medias_estudantes WHERE status = 'Aprovado';

-- Atualizar view
CREATE OR REPLACE VIEW vw_medias_estudantes AS ... ;

-- Remover view
DROP VIEW IF EXISTS vw_medias_estudantes;
```